13. 실제 사용 예제 및 코드 분석

개요

iroh-blob의 examples 디렉토리는 실제 사용 사례를 보여주는 6개의 예제를 포함하고 있습니다. 각 예제는 특정 시나리오에서 iroh-blob을 어떻게 활용할 수 있는지를 보여주며, BlobsProtocol을 이용한 서버 설정부터 클라이언트에서의 데이터 요청과 수신까지의 전체 과정을 다룹니다.

예제 분석

1. transfer.rs - 기본 파일 전송 예제

목적

가장 기본적인 파일 전송 시나리오를 구현한 예제로, 한 노드에서 다른 노드로 파일을 전송하는 과정을 보여줍니다.

핵심 구현 분석

서버 측 설정 (send 모드)

```
// 엔드포인트 생성 - P2P 네트워크의 진입점
let endpoint = Endpoint::builder().discovery_n0().bind().await?;


// 메모리 저장소 초기화
let store = MemStore::new();


// BlobsProtocol 초기화 - 블롭 요청을 처리하는 프로토콜 핸들러
```

```
let blobs = BlobsProtocol::new(&store, endpoint.clone(), None);


// 파일을 저장소에 추가하고 태그 생성
let tag = store.blobs().add_path(abs_path).await?;


// 티켓 생성 - 클라이언트가 파일을 요청할 때 사용
let ticket = BlobTicket::new(node_id.into(), tag.hash, tag.format);


// 라우터 설정 - 들어오는 연결을 적절한 프로토콜 핸들러로 라우팅
let router = Router::builder(endpoint)
.accept(iroh_blobs::ALPN, blobs)
.spawn();
```

클라이언트 측 구현 (receive 모드)

```
// 다운로더 생성 - 다른 노드로부터 파일을 다운로드하는 컴포넌트
let downloader = store.downloader(&endpoint);


// 파일 다운로드 실행
downloader
.download(ticket.hash(), Some(ticket.node_addr().node_id))
.await?;
```

```
// 다운로드된 파일을 지정된 경로로 내보내기
store.blobs().export(ticket.hash(), abs_path).await?;
```

주요 학습 포인트

- BlobsProtocol의 기본 설정 방법
- 파일을 블롭으로 변환하는 과정
- 티켓 시스템을 통한 콘텐츠 주소 지정
- 라우터를 통한 프로토콜 핸들링

2. get-blob.rs - 저장소 없는 블롭 요청

목적

저장소를 사용하지 않고 직접 원격 노드에서 블롭을 스트리밍으로 받아오는 방법을 보여줍니다.

핵심 구현 분석

직접 연결 및 스트리밍

```
// 엔드포인트 생성 (클라이언트 전용)
let endpoint = iroh::Endpoint::builder()
.discovery(PkarrResolver::n0_dns())
.bind()
.await?;

// 직접 연결 설정
let connection = endpoint
```

```
.connect(ticket.node_addr().node_id, iroh_blobs::ALPN)
.await?;


// 블롭 요청 시작
let mut progress = iroh_blobs::get::request::get_blob(connection, ticket.hash(
```

스트리밍 데이터 처리

```
loop {
match progress.next().await {
Some(GetBlobItem::Item(item)) => match item {
BaoContentItem::Leaf(leaf) => {
// 실제 데이터 청크를 즉시 출력
tokio::io::stdout().write_all(&leaf.data).await?;
}
BaoContentItem::Parent(parent) => {
// 메타데이터 정보 로깅
tracing::info!("Parent: {parent:?}");
}
},
Some(GetBlobItem::Done(stats)) => {
// 전송 완료
break stats;
}
```

```
Some(GetBlobItem::Error(err)) => {
// 오류 처리
anyhow::bail!("Error while streaming blob: {err}");
}
None => {
anyhow::bail!("Stream ended unexpectedly.");
}
}
}
```

주요 학습 포인트

- 저장소 없는 직접 스트리밍 방식
- BAO 콘텐츠 아이템의 구조와 처리
- 실시간 데이터 처리와 검증
- 진행 상황 추적 메커니즘

3. custom-protocol.rs - 커스텀 프로토콜 구현

목적

iroh-blob과 함께 커스텀 프로토콜을 구현하여 텍스트 검색 기능을 제공하는 복합 시스템을 보여줍니다.

핵심 구현 분석

커스텀 프로토콜 정의

```
// 커스텀 ALPN 정의
const ALPN: &[u8] = b"iroh-example/text-search/0";


#[derive(Debug, Clone)]
struct BlobSearch {
blobs: Store,
index: Arc<Mutex<HashMap<String, Hash>>>,
}
```

프로토콜 핸들러 구현

```
impl ProtocolHandler for BlobSearch {
async fn accept(&self, connection: Connection) -> std::result::Result<(), Acce
let (mut send, mut recv) = connection.accept_bi().await?;
// 쿼리 읽기
let query_bytes = recv.read_to_end(64).await.map_err(AcceptError::from_err)?;
let query = String::from_utf8(query_bytes).map_err(AcceptError::from_err)?;
// 로컬 검색 수행
let hashes = this.query_local(&query);
// 결과 해시들을 순차적으로 전송
for hash in hashes {
send.write_all(hash.as_bytes()).await.map_err(AcceptError::from_err)?;
}
```

```
send.finish()?;

Ok(())

}

}
```

다중 프로토콜 라우터 설정

```
// 커스텀 프로토콜과 iroh-blobs 프로토콜을 모두 지원

let  node = Router::builder(endpoint)

.accept(ALPN, proto.clone()) // 커스텀 검색 프로토콜

.accept(iroh_blobs::ALPN, blobs.clone()) // 표준 블롭 프로토콜

.spawn();
```

주요 학습 포인트

- 커스텀 프로토콜과 iroh-blob의 통합
- 다중 프로토콜 지원 라우터 구성
- 양방향 스트림을 이용한 요청-응답 패턴
- 프로토콜 간 데이터 공유 메커니즘

4. [mdns-discovery.rs](#) - 로컬 네트워크 발견

목적

mDNS를 이용한 로컬 네트워크에서의 노드 발견과 파일 전송을 구현합니다.

핵심 구현 분석

mDNS 발견 설정

```
// mDNS 발견 기능이 포함된 엔드포인트 생성
let endpoint = Endpoint::builder()
.secret_key(key)
.add_discovery(MdnsDiscovery::builder())
.relay_mode(RelayMode::Disabled) // 릴레이 서버 비활성화
.bind()
.await?;
```

로컬 파일 제공 (accept 모드)

```
// 파일을 저장소에 추가
let tag = store.add_path(absolute).await?;

// 연결 정보 출력
println!("To fetch the blob:\n\tcargo run --example mdns-discovery --features ex
node.endpoint().node_id(), tag.hash);
```

로컬 네트워크에서 다운로드 (connect 모드)

```
// 직접 연결 설정
let conn = endpoint.connect(node_id, iroh_blobs::ALPN).await?;

// 원격 fetch 수행
let stats = store.remote().fetch(conn, hash).await?;
```

```
// 파일로 내보내기
let size = store.export(hash, absolute).await?;
```

주요 학습 포인트

- mDNS를 통한 로컬 네트워크 발견
- 릴레이 서버 없는 직접 연결
- 로컬 네트워크 환경에서의 최적화
- 간단한 파일 공유 시스템 구현

5. [expiring-tags.rs](#) - 만료 태그 시스템

목적

시간 기반으로 만료되는 태그 시스템을 구현하여 자동 가비지 컬렉션을 보여줍니다.

핵심 구현 분석

만료 태그 생성

```
async fn create_expiring_tag(
store: &Store,
hashes: &[Hash],
prefix: &str,
expiry: SystemTime,
) -> anyhow::Result<()> {
// 만료 시간을 ISO8601 형식으로 변환
```

```
let expiry = chrono::DateTime::<chrono::Utc>::from(expiry);
let expiry = expiry.to_rfc3339_opts(chrono::SecondsFormat::Secs, true);
let tagname = format!("{prefix}-{expiry}");
if hashes.len() == 1 {
// 단일 해시의 경우
store.tags().set(&tagname, hashes[0]).await?;
} else {
// 다중 해시의 경우 HashSeq 사용
let hs = hashes.iter().copied().collect::<HashSeq>();
store
.add_bytes_with_opts(AddBytesOptions {
data: hs.into(),
format: BlobFormat::HashSeq,
})
.with_named_tag(&tagname)
.await?;
}
Ok(())
}
```

만료된 태그 삭제

```
async fn delete_expired_tags(blobs: &Store, prefix: &str, bulk: bool) -> anyho
```

```
let prefix = format!("{prefix}-");
let now = chrono::Utc::now();
let end = format!("{}-{}", prefix, now.to_rfc3339_opts(chrono::SecondsFormat::S
if bulk {
// 범위 삭제를 통한 효율적인 일괄 삭제
blobs
.tags()
.delete_range(Tag::from(prefix.clone())..Tag::from(end))
.await?;
} else {
// 개별 태그 확인 후 삭제
let mut tags = blobs.tags().list().await?;
let mut to_delete = Vec::new();
while let Some(tag) = tags.next().await {
let tag = tag?.name;
if let Some(rest) = tag.0.strip_prefix(prefix.as_bytes()) {
let expiry = std::str::from_utf8(rest)?;
let expiry = chrono::DateTime::parse_from_rfc3339(expiry)?;
if expiry.with_timezone(&Utc) < now {
to_delete.push(tag);
}
}
}
```

```
for tag in to_delete {
blobs.tags().delete(tag).await?;
}
}
Ok(())
}
```

주요 학습 포인트

- 시간 기반 태그 명명 규칙
- 자동 가비지 컬렉션 구현
- 범위 기반 태그 삭제 최적화
- HashSeq를 이용한 다중 해시 보호

6. random_store.rs - 대용량 데이터 테스트

목적

대량의 랜덤 데이터를 생성하고 다중 노드 간 전송을 테스트하는 시나리오를 구현합니다.

핵심 구현 분석

랜덤 데이터 생성 및 제공

```
// 파일 시스템 저장소 사용
let store = FsStore::load(&path).await?;

// 랜덤 블롭 생성
```

```
let blobs = create_random_blobs(
&store,
args.num_blobs,
|_, rand| rand.gen_range(1..=args.blob_size),
&mut rng,
).await?;


// 해시 시퀀스 생성
let hs = add_hash_sequences(
&store,
&blobs,
args.hash_seqs,
|_, rand| rand.gen_range(1..=args.hash_seq_size),
&mut rng,
).await?;
```

이벤트 기반 모니터링

```
pub fn dump_provider_events(allow_push: bool) -> (
tokio::task::JoinHandle<()>,
mpsc::Sender<iroh_blobs::provider::Event>,
) {
let (tx, mut rx) = mpsc::channel(100);
let dump_task = tokio::spawn(async move {
```

```
while let Some(event) = rx.recv().await {
match event {
Event::ClientConnected { node_id, connection_id, permitted } => {
permitted.send(true).await.ok();
println!("Client connected: {node_id} {connection_id}");
}
Event::GetRequestReceived { connection_id, request_id, hash, ranges } => {
println!("Get request received: {connection_id} {request_id} {hash} {ranges:?}")
}
Event::TransferCompleted { connection_id, request_id, stats } => {
println!("Transfer completed: {connection_id} {request_id} {stats:?}");
}
// ... 기타 이벤트 처리
}
}
});
(dump_task, tx)
}
```

다중 소스 다운로드

```
// 셔플된 노드 목록으로 다운로드
for content in args.content {
```

```
let mut progress = downloader
.download(content, Shuffled::new(nodes.clone()))
.stream()
.await?;
while let Some(event) = progress.next().await {
info!("Progress: {:?}", event);
}
}
```

주요 학습 포인트

- 대용량 데이터 처리 패턴
- 이벤트 기반 모니터링 시스템
- 다중 소스 다운로드 전략
- 성능 테스트 및 벤치마킹

BlobsProtocol 서버 설정 과정

1. 기본 설정 패턴

모든 예제에서 공통적으로 사용되는 BlobsProtocol 설정 패턴:

```
// 1. 엔드포인트 생성
let endpoint = Endpoint::builder()
.secret_key(secret_key) // 노드 식별을 위한 비밀키
.discovery_n0() // 노드 발견 서비스 활성화
```

```
.bind()
.await?;


// 2. 저장소 초기화
let store = MemStore::new(); // 또는 FsStore::load(&path)


// 3. BlobsProtocol 생성
let blobs = BlobsProtocol::new(
&store, // 저장소 참조
endpoint.clone(), // 엔드포인트 참조
Some(events_tx) // 선택적 이벤트 채널
);


// 4. 라우터 설정 및 시작
let router = Router::builder(endpoint)
.accept(iroh_blobs::ALPN, blobs) // ALPN 프로토콜 등록
.spawn();
```

2. 고급 설정 옵션

이벤트 모니터링 설정

```
let (dump_task, events_tx) = dump_provider_events(allow_push);
let blobs = BlobsProtocol::new(&store, endpoint.clone(), Some(events_tx));
```

다중 프로토콜 지원

```
let router = Router::builder(endpoint)
.accept(CUSTOM_ALPN, custom_protocol) // 커스텀 프로토콜
.accept(iroh_blobs::ALPN, blobs) // 표준 블롭 프로토콜
.spawn();
```

발견 서비스 커스터마이징

```
let endpoint = Endpoint::builder()
.add_discovery(MdnsDiscovery::builder()) // mDNS 발견
.add_discovery(PkarrResolver::n0_dns()) // DNS 기반 발견
.relay_mode(RelayMode::Disabled) // 릴레이 비활성화
.bind()
.await?;
```

클라이언트 데이터 요청 및 수신 과정

1. 기본 다운로드 패턴

Store API를 통한 다운로드

```
// 1. 다운로더 생성
let downloader = store.downloader(&endpoint);
```

```
// 2. 다운로드 실행
downloader
.download(hash, Some(node_id))
.await?;


// 3. 데이터 접근
let data = store.get_bytes(hash).await?;
```

직접 연결을 통한 스트리밍

```
// 1. 연결 설정
let connection = endpoint.connect(node_id, iroh_blobs::ALPN).await?;


// 2. 블롭 요청 시작
let mut progress = iroh_blobs::get::request::get_blob(connection, hash);


// 3. 스트리밍 처리
while let Some(item) = progress.next().await {
match item {
GetBlobItem::Item(BaoContentItem::Leaf(leaf)) => {
// 데이터 청크 처리
process_data(&leaf.data);
}
GetBlobItem::Done(stats) => {
```

```
// 완료 처리
break;
}
GetBlobItem::Error(err) => {
// 오류 처리
return Err(err.into());
}
}
}
```

2. 고급 다운로드 패턴

다중 소스 다운로드

```
let nodes = vec![node1, node2, node3];
let mut progress = downloader
.download(content, Shuffled::new(nodes))
.stream()
.await?;

while let Some(event) = progress.next().await {
match event {
DownloadEvent::Progress { hash, progress } => {
println!("Progress for {}: {:.1}%", hash.fmt_short(), progress * 100.0);
```

```
}
DownloadEvent::Done { hash, stats } => {
println!("Completed {}: {} bytes", hash.fmt_short(), stats.payload_bytes_read);
}
}
}
```

부분 다운로드

```
// 특정 범위만 요청
let ranges = ChunkRanges::from(0..100); // 처음 100개 청크만
let request = GetRequest {
hash,
ranges: ranges.into(),
};

let connection = endpoint.connect(node_id, iroh_blobs::ALPN).await?;
let mut stream = get::request::get_blob_ranges(connection, request);
```

시나리오별 최적 사용 패턴

1. 간단한 파일 공유 (transfer.rs 패턴)

적용 시나리오:

- 개인 간 파일 전송
- 일회성 데이터 공유
- 소규모 파일 배포

최적화 포인트:

- MemStore 사용으로 빠른 처리
- 간단한 티켓 시스템
- 최소한의 설정

2. 실시간 스트리밍 (get-blob.rs 패턴)

적용 시나리오:

- 대용량 파일의 실시간 처리
- 메모리 사용량 최소화 필요
- 스트리밍 미디어 전송

최적화 포인트:

- 저장소 없는 직접 스트리밍
- 청크 단위 실시간 처리
- 메모리 효율성

3. 복합 서비스 (custom-protocol.rs 패턴)

적용 시나리오:

- 검색 기능이 포함된 파일 서버
- 메타데이터 기반 콘텐츠 발견
- 다중 프로토콜 지원 필요

최적화 포인트:

- 커스텀 프로토콜과 표준 프로토콜 조합
- 효율적인 인덱싱 시스템
- 프로토콜 간 데이터 공유

4. 로컬 네트워크 공유 (mdns-discovery.rs 패턴)

적용 시나리오:

- 사무실 내 파일 공유
- 로컬 네트워크 백업
- 오프라인 환경에서의 데이터 동기화

최적화 포인트:

- mDNS를 통한 자동 발견
- 릴레이 서버 불필요
- 네트워크 대역폭 최적화

5. 자동 정리 시스템 (expiring-tags.rs 패턴)

적용 시나리오:

- 임시 파일 관리
- 캐시 시스템
- 자동 백업 정리

최적화 포인트:

- 시간 기반 자동 정리
- 효율적인 범위 삭제
- 가비지 컬렉션 최적화

6. 대규모 데이터 처리 (random_store.rs 패턴)

적용 시나리오:

- 대용량 데이터 배포
- 성능 테스트 및 벤치마킹
- 다중 소스 동기화

최적화 포인트:

- 파일 시스템 저장소 활용
- 이벤트 기반 모니터링
- 다중 소스 병렬 처리

공통 유틸리티 및 헬퍼 함수

1. 공통 설정 (common/mod.rs)

비밀키 관리

```
pub fn get_or_generate_secret_key() -> Result<SecretKey> {
if let Ok(secret) = env::var("IROH_SECRET") {
SecretKey::from_str(&secret).context("Invalid secret key format")
} else {
let secret_key = SecretKey::generate(&mut thread_rng());
println!("Generated new secret key: {}", hex::encode(secret_key.to_bytes()));
Ok(secret_key)
}
```

```
}
```

로깅 설정

```
pub fn setup_logging() {
tracing_subscriber::registry()
.with(tracing_subscriber::fmt::layer().with_writer(std::io::stderr))
.with(EnvFilter::from_default_env())
.try_init()
.ok();
}
```

2. 오류 처리 패턴

일관된 오류 처리

```
match progress.next().await {
Some(GetBlobItem::Item(item)) => {
// 정상 처리
}
Some(GetBlobItem::Error(err)) => {
anyhow::bail!("Error while streaming blob: {err}");
}
None => {
anyhow::bail!("Stream ended unexpectedly.");
```

```
}

}
```

연결 오류 복구

```
let  connection = endpoint

.connect(node_id, iroh_blobs::ALPN)

.await

.context("Failed to connect to remote node")?;
```

성능 최적화 기법

1. 메모리 사용량 최적화

- **스트리밍 처리**: 전체 파일을 메모리에 로드하지 않고 청크 단위로 처리
- **적응적 저장소**: 작은 파일은 메모리, 큰 파일은 디스크 사용
- **참조 카운팅**: 중복 데이터의 효율적 관리

2. 네트워크 최적화

- **다중 스트림**: QUIC의 다중 스트림 기능 활용
- **연결 재사용**: 동일 노드에 대한 연결 재사용
- **적응적 청크 크기**: 네트워크 상황에 따른 청크 크기 조정

3. I/O 최적화

- **비동기 처리**: tokio 기반 비동기 I/O

- **배치 처리**: 여러 작업을 배치로 처리
- **캐싱**: 자주 사용되는 데이터의 캐싱

결론

iroh-blob의 예제들은 다양한 사용 시나리오에서 어떻게 시스템을 활용할 수 있는지를 보여줍니다. 각 예제는 특정 요구사항에 최적화된 패턴을 제시하며, 실제 프로덕션 환경에서 참고할 수 있는 모범 사례를 포함하고 있습니다. BlobsProtocol의 유연성과 확장성을 통해 단순한 파일 전송부터 복잡한 분산 시스템까지 다양한 애플리케이션을 구축할 수 있음을 확인할 수 있습니다.